

Parte 3 do EP 2

11 de julho de 2026

1 Estado da Arte

De acordo com (Pan; Wang; Li, 2023), atualmente, existem mais de 20 database vector systems no mercado. Vamos focar nos mais modernos e usados atualmente no mercado e no ambiente científico.

2 Pinecone

A evolução recente dos *Vector Database Management Systems* é marcada pela transição de índices mantidos estritamente em memória para arquiteturas nativas em nuvem e sem servidor. Neste cenário, o **Pinecone** destaca-se, projetado especificamente para suprir as demandas de latência e consistência de aplicações baseadas em LLMs e RAG. (Zhan; Kang *et al.*, 2025).

O Pinecone, diferente dos próximos que serão apresentados, é de código fechado e é um SaaS, isto é, o usuário não se preocupa em gerenciar o sistema, a escalabilidade, nem a infraestrutura. Porém, isso vem com um alto custo monetário e o usuário fica preso ao sistema de nuvem do Pinecone.

2.1 Arquitetura Desacoplada e Armazenamento

O sistema de armazenamento utiliza estruturas de arquivos imutáveis persistidos em serviços de armazenamento de objetos distribuídos. A execução de consultas é delegada a executores, permitindo a auto-escalabilidade horizontal e independente do throughput de leitura (*QPS*) e do volume total de vetores armazenados, mitigando os custos operacionais de infraestrutura em escala de bilhões de vetores.

2.2 Busca Híbrida e Filtragem de Metadados em Etapa Única

Um dos principais gargalos teóricos em *Vector Database Management Systems* é a eficiência em buscas vetoriais restritas por atributos escalares (filtros de metadados). As abordagens tradicionais dividem-se em:

1. **Pré-filtragem:** Reduz o espaço de busca aplicando o filtro escalar antes da navegação no grafo ANN, o que frequentemente fragmenta a topologia do grafo e degrada o retorno dos dados.

2. **Pós-filtragem:** Executa a busca vetorial no grafo completo e descarta os resultados que não satisfazem o filtro escalar, resultando em latência desproporcional e retrabalho computacional caso a seletividade do filtro seja alta.

O Pinecone resolve esta limitação através do mecanismo de **Filtragem em Etapa Única**, onde índices invertidos e vetores de bits de metadados são integrados diretamente ao processo de travessia do algoritmo de Vizinhos Mais Próximos Aproximados (ANN). Esse fluxo unificado garante latência determinística $O(\log N)$ mesmo em cenários com filtros altamente seletivos e combinados com representações esparsas e densas simultaneamente.

3 Milvus

Diferentemente do Pinecone, o Milvus é um sistema gerenciador de código aberto. Amplamente estudado e usado no meio científico e empresarial.

3.1 Desacoplamento de Atores e Ciclo de Vida dos Dados

A inovação estrutural do Milvus reside na separação estrita entre computação, armazenamento e fluxo de mensagens de log. O sistema divide suas operações em três camadas independentes:

1. **Camada de Coordenação:** Funciona como o cérebro do cluster, gerenciando a atribuição de nós, topologia da rede, balanceamento de carga e metadados do sistema através de um motor de consenso distribuído.
2. **Camada de Execução:** Composta por nós trabalhadores especializados e *stateless*. Subdivide-se em *Query Nodes*, exclusivos para computação de consultas ANN, *Data Nodes*, responsáveis pelo consumo de logs e persistência e *Index Nodes*, dedicados à construção assíncrona de índices vetoriais pesados.
3. **Camada de Mensageria e Armazenamento:** Utiliza corretores de mensagens, como Apache Pulsar ou Kafka, para garantir a persistência de logs de escrita (*Write-Ahead Logging* - WAL) antes que os dados sejam armazenados em sistemas como Amazon S3.

3.1.1 Consistência Dinâmica e Segmentação de Dados Semânticos

Diferente dos bancos relacionais ACID clássicos, o Milvus permite ao usuário parametrizar o nível de consistência em tempo de execução, variando desde a consistência forte até a consistência eventual (Wang *et al.*, 2021). Para processar o fluxo contínuo de dados, o sistema introduz os conceitos de dados *Bounded*, segmentos fechados e indexados e *Unbounded*, dados recém-inseridos que residem em buffers de memória.

Durante uma consulta, o Milvus executa uma busca paralela híbrida. Temos o conjunto de vetores já consolidados em índices estruturados e os vetores em estado de *stream* ainda não indexados. A operação de busca de vizinhos mais próximos é formalizada como a união dos resultados computados sobre ambas as partições de memória.

Onde o KNN executa uma busca exaustiva de força bruta via força computacional paralela, usando núcleos CUDA em GPUs, se disponíveis, garantindo que dados recém-inseridos fiquem disponíveis para consulta em milissegundos sem interromper a estabilidade dos índices globais de larga escala (Wang *et al.*, 2021).

4 pgvector

Diferentemente do Milvus e do Pinecone, o pgvector não foi construído do zero como um sistema puramente vetorial. Mas sim, foi construído como uma extensão do Postgres. Dessa forma, transformou o PostgreSQL em um banco de dados que suporta vetorial e relacional.

4.1 Mecanismos de Funcionamento e Indexação

A extensão introduziu o tipo nativo `vector` e operadores geométricos fundamentais para calcular a distância entre um vetor de consulta $\vec{q}$ e um vetor armazenado $\vec{x}$. É utilizado cálculo de distância simples já mencionados anteriormente, como: distância euclidiana e a similaridade de cossenos.

4.2 Algoritmos de Indexação Suportados

Para mitigar a complexidade exaustiva $\mathcal{O}(N)$, o pgvector implementa dois algoritmos principais de Busca por Vizinhos Próximos Aproximados (*Approximate Nearest Neighbor Search* - *ANNS*):

1. **IVFFlat:** Executa a clusterização do espaço vetorial através de k -means criando listas invertidas. Possui baixo consumo de memória e rápida construção, mas menor precisão se o número de partições examinadas for pequeno.
2. **HNSW:** Constrói um grafo hierárquico estruturado de múltiplos níveis. Possui uma ótima precisão e um ótimo tempo, embora demande maior processamento e memória RAM para a construção e manutenção do índice.

4.3 Vantagens:

- **Consistência ACID Completa:** O vetor e os dados transacionais residem no mesmo bloco de dados, garantindo atomicidade absoluta via logs de escrita antecipada (WAL).
- **Filtragem Híbrida Eficiente:** O otimizador de consultas do PostgreSQL avalia estatísticas estruturadas e índices vetoriais no mesmo plano de execução, permitindo instruções SQL complexas

4.4 Limitações Estruturais do pgvector

Apesar de sua enorme conveniência, a arquitetura baseada em extensão impõe gargalos físicos rígidos que impedem o pgvector de substituir sistemas massivamente distribuídos (como o Milvus) em cenários de hiperescala.

4.4.1 Gargalos e Restrições Técnicas

1. Limite de Dimensões: o que restringe o número de dimensões do embedding para 2.000, impedindo o uso de modelos científicos pesados de última geração sem a aplicação de técnicas de compressão.

2. Dependência de CPU: Depende inteiramente de instruções SIMD de CPU. Sistemas como o Milvus utilizam paralelismo massivo em GPU (Nvidia CUDA), operando ordens de magnitude acima do rendimento suportado pelo PostgreSQL em grandes volumes.
3. Acoplamento de recursos: Como o PostgreSQL opera em uma arquitetura baseada em processos estruturados para CPUs, cálculos vetoriais intensivos competem diretamente por CPU e memória RAM com transações críticas de leitura/escrita. Um pico de buscas semânticas pode exaurir os recursos e degradar a aplicação principal.

5 Qdrant

O **Qdrant** é um banco de dados de vetores projetado para buscas de vizinhos mais próximos estruturadas (*Approximate Nearest Neighbor - ANN*) com suporte nativo a filtragem de metadados. Desenvolvido em Rust, seu diferencial reside na eficiência de memória e na execução de filtros em estágio único.

5.1 Estrutura de Dados Básica

No Qdrant, a unidade fundamental de armazenamento é um ponto. Matematicamente, um ponto P é definido como uma tupla:

$$P = (id, \vec{v}, \text{payload}) \quad (1)$$

Onde:

- $id \in N$ ou UUID é o identificador único do ponto.
- $\vec{v} \in R^d$ é o vetor de alta dimensão (d). O Qdrant também suporta múltiplos vetores nomeados por ponto: $\vec{v}_1, \vec{v}_2, \dots, \vec{v}_n$.
- payload é um objeto JSON que armazena os metadados associados (ex: marcas, preços, strings).

5.2 Métricas de Distância

Para determinar a similaridade entre o vetor de busca $\vec{q}$ e um vetor armazenado $\vec{v}$, o Qdrant calcula a distância matemática baseada na métrica escolhida na criação da coleção: como similaridade de cossenos, distância euclidiana, entre outros

5.3 O Índice de Busca: HNSW

Para evitar a busca linear $O(N)$, o Qdrant utiliza uma variação do algoritmo **HNSW** (*Hierarchical Navigable Small World*).

O HNSW constrói uma estrutura de grafos em múltiplas camadas .

1. A **Camada Superior** (L_m) possui poucos nós e conexões longas para buscas rápidas e grosseiras.
2. À medida que se desce nas camadas ($L_{m-1} \dots L_0$), o grafo se torna mais denso, permitindo o refinamento da busca até encontrar os vizinhos mais próximos na L_0 , que contém todos os vetores.

5.4 Filtragem em Estágio Único

O maior diferencial do Qdrant é como ele une a busca vetorial com os filtros de *payload*. Em vez de filtrar antes ou depois da busca vetorial, o Qdrant faz a **filtragem dinâmica durante a travessia do grafo HNSW**.

Uma condição de filtro definida pelo usuário, por exemplo: `preco < 50`). Durante a busca no grafo, ao avaliar se deve avançar para um nó vizinho V_j , o algoritmo valida: se o *payload* do V_j satisfaz a condição do usuário

Se o nó não satisfizer a condição, a aresta é ignorada no cálculo de proximidade vetorial, garantindo que o resultado final seja matematicamente preciso e respeite estritamente as regras de negócio sem perder performance.

5.5 Otimização de Memória: Quantização

Para mitigar o alto consumo de RAM, o Qdrant implementa a Quantização Escalar. Ela converte os valores de ponto flutuante de 32 bits para inteiros de 8 bits, técnica similar a empregada em modelos de Deep Learning.

Isso reduz o tamanho do índice em até $4\times$, permitindo que bilhões de dimensões sejam processadas com eficiência de cache de CPU e consumo de memória drasticamente menor.

6 Análise Comparativa

A tabela apresenta uma consolidação visual e técnica dos principais sistemas gerenciadores de bancos de dados vetoriais analisados neste relatório.

Tabela 1: Comparativo: Linguagem Base e Algoritmos de Indexação

Sistema	Linguagem Base	Algoritmos de Indexação Suportados
Pinecone	Go / C++	Proprietário (Estrutura de Grafo / ANN).
Milvus	Go / C++ / Python	HNSW, IVF, ScaNN, ANNOY.
pgvector	C (Extensão Postgres)	HNSW, IVFFlat.
Qdrant	Rust	HNSW customizado.

Tabela 2: Comparativo: Suporte a Busca Híbrida e Tipo de Licença

Sistema	Busca Híbrida (Vetor + Metadados)	Tipo de Licença
Pinecone	Filtragem em Etapa Única nativa com vetores de bits.	Proprietária (SaaS)
Milvus	Filtragem dinâmica paralela sobre dados indexados e em <i>stream</i> .	Open Source
pgvector	Filtros relacionais e vetoriais integrados no otimizador SQL.	Open Source
Qdrant	Filtragem dinâmica em estágio único durante a travessia do grafo.	Open Source

Tabela 3: Comparativo: Modelo de Escalabilidade

Sistema	Modelo de Escalabilidade
Pinecone	Nuvem nativa, totalmente desacoplada e sem servidor (<i>Serverless</i>), focada em auto-escalabilidade horizontal.
Milvus	Altamente distribuído com separação estrita de três camadas independentes (Coordenação, Execução e Mensageria).
pgvector	Essencialmente vertical, operando acoplado diretamente aos recursos físicos e processos de CPU/RAM da instância PostgreSQL.
Qdrant	Escalabilidade horizontal nativa através de um sistema robusto e eficiente de <i>sharding</i> construído em Rust.

Tabela 4: Comparativo: Vantagens e Desvantagens dos Sistemas

Sistema	Vantagens	Desvantagens
Pinecone	Totalmente gerenciado (SaaS); infraestrutura e escalabilidade transparentes; excelente para RAG com filtragem em etapa única nativa.	Código fechado (proprietário); alto custo financeiro em larga escala; dependência total do ecossistema de nuvem do provedor (<i>vendor lock-in</i>).
Milvus	Código aberto; altamente distribuído e robusto; permite consistência dinâmica e paralelismo massivo utilizando hardware focado (GPUs/CUDA).	Arquitetura complexa com muitas partes móveis (três camadas distintas); curva de aprendizado e manutenção de infraestrutura elevadas.
pgvector	Consistência ACID completa; integração nativa com dados relacionais existentes no PostgreSQL; sem necessidade de gerenciar um novo banco de dados.	Limite físico de 2.000 dimensões; dependência estrita de CPU (sem aceleração por GPU); cálculos vetoriais intensivos competem por recursos com transações SQL críticas.
Qdrant	Desenvolvido em Rust (alta eficiência de memória); filtragem dinâmica em estágio único durante a travessia do grafo HNSW; quantização escalar nativa.	Comunidade e ecossistema de ferramentas integradas ligeiramente menores quando comparados ao Milvus ou soluções de mercado mais antigas.

7 Referências

Referências

PAN, James Jie; WANG, Jianguo; LI, Guoliang. **Survey of Vector Database Management Systems**. [*S. l.*: *s. n.*], 2023. arXiv: 2310.14021 [cs.DB]. Disponível em: <https://arxiv.org/abs/2310.14021>.

WANG, Jianguo *et al.* Milvus: A Purpose-Built Vector Data Management System. Association for Computing Machinery, Virtual Event, China, p. 2614–2627, 2021. DOI: 10.1145/3448016.3457550. Disponível em: <https://doi.org/10.1145/3448016.3457550>.

ZHAN, Guang; KANG, Gao *et al.* BigVectorBench: Heterogeneous Data Embedding and Compound Queries are Essential in Evaluating Vector Databases. **Proceedings of the VLDB Endowment**, v. 18, n. 5, p. 1120–1133, 2025.